Debugging with Blockly

Finding out why the robot did not do what you meant.

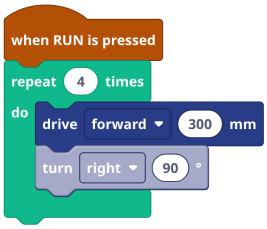
A **bug** is the difference between what you think the program does and what it really does. Do not guess: make the program **tell you**. It talks in the **Console**, next to the simulator. Cards marked TRY THIS fix the most bugs.

1 Read the console

```
says: 1: to the wall
sim: keep driving   v40,0,0;
sim: stop   x
says: 2: turn around
sim: turn right 180°   m0,0,180;
says: 3: back
```

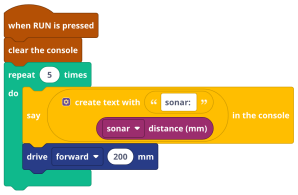
says: is your **say** block, **sim:** a move. ⚠ is a warning, ✖ an error: the program stopped there.

2 Watch it run



The block that is running **lights up**. Set the simulator **Speed** to **real time** to follow along.

3 Say what it sees

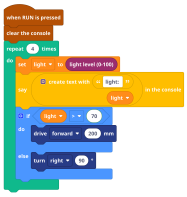



```
says: sonar: 1585
sim: forward 200 mm   m0,200,0;
says: sonar: 1385
```

Put **say** inside the loop: the console shows the sensor **while** the program runs.

4 Read once, use it twice

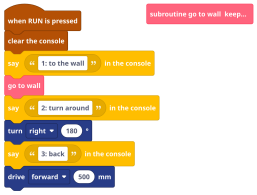
TRY THIS



```
says: light: 66
sim: turn right 90°   m0,0,90;
```

Every sensor block reads **again**. Read it into a variable once, then **say** that variable and **decide** with it: the number you see is the number it used.

5 Label every say

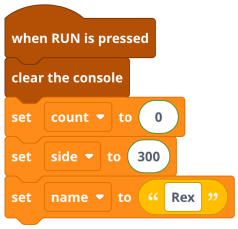



```
says: 2: turn around
says: 3: back
```

A bare **412** could be anything: say **"wall: "** and the number. Number your steps: the last one in the console is how far it got.

6 Start clean

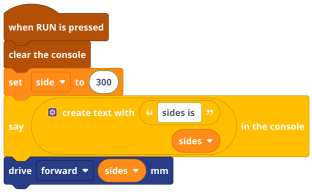
TRY THIS



Right under **when RUN**: **clear the console**, then **set** every variable. Each run starts the same. (The **Clear** button empties it by hand.)

7 "undefined" means never set

TRY THIS




```
says: sides is undefined
Program finished.
```

Every RUN starts all variables **empty**. **sides** is not **side**: a second variable nobody set. And the drive? **Skipped**, with no warning.

8 Simulator first

Simulator | Robot

Wrong in the **simulator** too? It is the program. Right in the simulator, wrong on the robot? Look at the robot: its numbers, battery, sensor.

9 One change at a time

1. Guess what is wrong.
2. Change **one** thing.
3. RUN.
4. Look: better? Keep it. Not? **Undo** (Ctrl+Z).

10 Make it smaller

Right-click, **Disable Block** on half the program. Bug gone? It is in that half. Or put just **one subroutine** under a **when RUN** and test it alone.

11 Run it three times

Maps **4 to 10** change every run, and real floors and batteries change too. Worked once? Maybe lucky. Three in a row: it works.

12 Explain it out loud

Read your program to a friend (or a toy), block by block: "then it turns right 90..." Bugs show up when you say what each block **really** does.

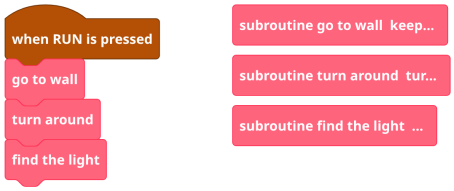
Organize it, then ask the robot

Programs that are easy to read are easy to fix. Then the real robot: what it sees, what it heard.

Cards marked ROBOT need a robot connected (**Serial** or **Bluetooth**) and **Run on: Robot**. Numbers on them are examples: yours will be different. That is the point of measuring.

13 One job per subroutine

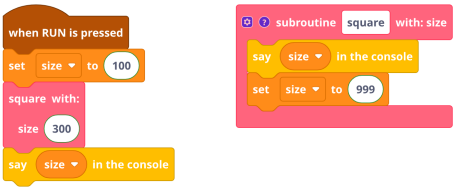
TRY THIS



Do not pack it all into one long stack. Give each job a subroutine with a **name that says what it does**. The main program reads like a list.

14 Inputs get their own names

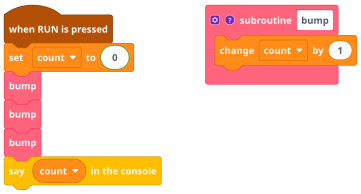
TRY THIS




```
says: 300
says: 100
```

The input **size** said 300, the program's **size** stayed 100: **two boxes**, one name. Do not reuse a name: call the input **square size**.

15 Subroutines share the rest




```
says: 3
```

Every **other** variable is shared: **bump** changes the program's **count**. Handy for counters. To make a subroutine work on **any** number, use an input.

16 Each tab is its own program

```
says: 5
says: 1
```

x in one tab and **x** in another are two boxes. **run program** runs the other tab, but its variables stay there: here it said 5, and ours is still 1.

17 Good names

wall, **side**, **light at start**: not **x** and **a**. A name that says what is inside makes the bug easy to spot.

18 Save a copy first

About to change a lot? **This program...**, **Duplicate**. Then try. Broken? The copy is still there. Small changes: **Ctrl+Z**.

19 Ask the robot

ROBOT

- **u** the sonar: `U,412` is 412 mm, `U,-1` nothing in range
- **b** the light sensors: `B,` direction, brightest, left, right (0 to 1000)
- **p?;** what is saved in the remote slots
- **x** stop the wheels
- **?** the list of every command

Connect, type in the box under the console, press **Send**.

20 Measure first

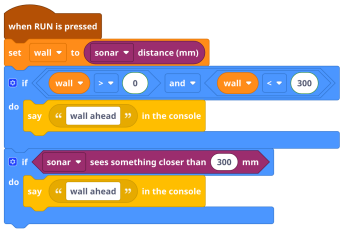
ROBOT

```
→ b
← B,0,412,398,412
→ b
← B,-13,846,846,520
```

Light numbers depend on the room. Read **b** where the robot will drive, then with the flashlight, and pick a number **in between**. The light level block is the brightest **÷ 10**.

21 Sonar -1 means nothing

ROBOT



-1: nothing in range (or no echo). **wall < 300** is true for -1! Check **wall > 0** too, or use **sees something closer than**: it ignores -1.

22 The arrows: sent and answered

ROBOT

```
→ m0,300,0;
← move: done
→ u
← U,412
says: wall: 412
```

With **Robot**: → is what the page sent, ← what the robot answered. No ← at all? The robot did not hear it.

23 The robot will do

1. forward 300 mm
2. turn right 90°
3. forward 300 mm
4. turn right 90°
5. say "half way"

From here on it depends on what the sonar sees, so the rest cannot be worked out ahead of time.

Check the plan **before** you RUN: every move, in order, up to the first sensor block.

24 Robot not doing it?

ROBOT

- Switch on, shield light bright (dim: batteries).
- **Run on: Robot**, and connected.
- One thing at a time over Bluetooth: wait for it, STOP if stuck.
- **u** and **b** answer? Sensor plugged in right.